Consolidated Monitoring Dashboard API Specification

Version: 1.0
Base URL: `https://myxxit-devopshq.onrender.com`
Authentication: Bearer Token (API Key)
Content-Type: `application/json`
Timeout: 5s per endpoint (see individual specs)

API Endpoints Quick Reference

Method	Endpoint	Purpose	Auth	Timeout
GET	`/api/monitoring/health`	Service health check	No	2s
POST	`/api/monitoring/refresh`	Trigger report refresh	Yes	10s
GET	`/api/monitoring/reports/{id}`	Fetch specific report	Yes	2s
GET	`/api/monitoring/reports`	List reports (paginated)	Yes	2s
POST	`/webhooks/hq-task-updated`	Task update webhook	Yes (Bearer)	5s
POST	`/webhooks/github-pr-event`	GitHub PR webhook	Yes (HMAC)	5s

Detailed Endpoint Specifications

1. GET /api/monitoring/health

Health check endpoint. Requires no authentication.

Use Cases: - Monitor service availability - Verify datasource connectivity (HQ, GitHub) - CI/CD pipeline health checks

Request:

```
GET /api/monitoring/health HTTP/1.1
Host: myxxit-devopshq.onrender.com
```

Response 200 OK (Healthy):

```
{
  "status": "healthy",
  "service": "monitoring-dashboard",
  "uptime_seconds": 86400,
  "timestamp": "2026-04-09T19:00:00Z",
  "datasources": {
    "hq": {
      "status": "online",
      "last_check": "2026-04-09T19:00:00Z",
      "response_time_ms": 120
    },
    "github": {
      "status": "online",
      "last_check": "2026-04-09T19:00:00Z",
      "response_time_ms": 200
    }
  }
}
```

Response 503 Unavailable (Degraded):

```
{
  "status": "degraded",
  "service": "monitoring-dashboard",
  "timestamp": "2026-04-09T19:00:00Z",
  "datasources": {
    "hq": {
      "status": "online",
      "response_time_ms": 150
    },
    "github": {
      "status": "offline",
      "last_error": "Connection timeout",
      "last_check": "2026-04-09T18:55:00Z"
    }
  },
  "message": "GitHub API temporarily unavailable; reports will be
        generated from cached data"
}
```

cURL Example:

```
curl -s https://myxxit-devopshq.onrender.com/api/monitoring/
        health | jq .
```

---

2. POST /api/monitoring/refresh

Trigger a consolidated report refresh. Returns report immediately or as background job.

Use Cases: - Manual “Refresh” button click in UI - Scheduled refresh (cron) - Webhook-triggered refresh (task updated, PR merged)

Request Headers:

```
POST /api/monitoring/refresh HTTP/1.1
Host: myxxit-devopshq.onrender.com
Authorization: Bearer YOUR_API_KEY
Content-Type: application/json
```

Request Body (Optional):

```
{
  "fullRefresh": false,
  "includeArchive": true,
  "maxAge": 300,
  "mode": "sync"
}
```

Parameters:

Field	Type	Default	Description
`fullRefresh`	boolean	false	Force fetch from APIs (ignore cache)
`includeArchive`	boolean	true	Save report to workspace markdown
`maxAge`	number	300	Return cached report if <N seconds old
`mode`	enum	“sync”	“sync” (wait) or “async” (background)

Response 200 OK (Sync Mode, Successful):

```
{
  "status": "success",
  "mode": "sync",
  "report": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "generatedAt": "2026-04-09T19:00:00Z",
```

```
"refreshSource": "manual",
"metrics": {
  "totalTasks": 12,
  "byStatus": {
    "proposed": 2,
    "inProgress": 5,
    "completed": 3,
    "blocked": 2
  },
  "blockedCount": 2,
  "riskSummary": {
    "high": 1,
    "medium": 2,
    "low": 9
  }
},
"tasks": [
  {
    "id": "task-001",
    "title": "Implement user authentication",
    "status": "in-progress",
    "owner": "alice@example.com",
    "riskLevel": "medium",
    "blockers": [
      {
        "id": "blocker-001",
        "title": "Database schema approval pending",
        "severity": "high",
        "blockedBy": "bob@example.com",
        "createdAt": "2026-04-08T10:00:00Z"
      }
    ],
    "linkedPR": {
      "number": 42,
      "url": "https://github.com/org/repo/pull/42",
      "status": "open",
      "prChanges": 3
    },
    "createdAt": "2026-04-01T00:00:00Z",
    "updatedAt": "2026-04-09T18:00:00Z",
    "tags": ["backend", "priority-high"]
  }
],
"blockers": [
  {
    "id": "blocker-001",
    "taskId": "task-001",
    "title": "Database schema approval pending",
    "description": "Waiting for DBA review of new
    user_preferences table",
    "severity": "high",
    "blockedBy": "bob@example.com",
```

```
        "owner": "alice@example.com",
        "createdAt": "2026-04-08T10:00:00Z"
      }
    ],
    "prStatus": [
      {
        "prNumber": 42,
        "title": "feat: add user authentication",
        "author": "alice",
        "status": "open",
        "linkedTaskId": "task-001",
        "reviewsRequired": 2,
        "reviewsGiven": 1,
        "hasConflicts": false,
        "createdAt": "2026-04-05T12:00:00Z",
        "commits": 3,
        "url": "https://github.com/org/repo/pull/42"
      }
    ],
    "errors": [],
    "datasources": {
      "hq": {
        "status": "success",
        "lastSync": "2026-04-09T19:00:00Z"
      },
      "github": {
        "status": "success",
        "lastSync": "2026-04-09T19:00:00Z"
      }
    },
    "persistedAt": "docs/reports/2026-04-09_1900_refresh.md"
  },
  "duration_ms": 1250,
  "cached": false
}
```


Response 200 OK (Cached Report):


```
{
  "status": "success",
  "mode": "sync",
  "report": { ... },
  "duration_ms": 15,
  "cached": true,
  "cacheAge_seconds": 45
}
```


Response 202 Accepted (Async Mode):


```
{
  "status": "accepted",
  "mode": "async",
```

```
  "reportId": "550e8400-e29b-41d4-a716-446655440000",
  "estimatedTime_ms": 2500,
  "checkStatusUrl": "/api/monitoring/reports/550e8400-e29b-41d4-
        a716-446655440000",
  "pollInterval_ms": 500
}
```

Response 400 Bad Request:

```
{
  "status": "error",
  "code": "INVALID_REQUEST",
  "message": "Invalid maxAge parameter (must be >= 0)",
  "details": {
    "field": "maxAge",
    "value": -100,
    "constraint": ">= 0"
  }
}
```

Response 401 Unauthorized:

```
{
  "status": "error",
  "code": "AUTH_FAILED",
  "message": "Invalid or missing API key",
  "hint": "Provide API key in Authorization header: Bearer
        <YOUR_API_KEY>"
}
```

Response 503 Service Unavailable (Degraded):

```
{
  "status": "degraded",
  "mode": "sync",
  "report": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "generatedAt": "2026-04-09T19:00:00Z",
    "tasks": [...],
    "blockers": [...],
    "errors": [
      {
        "source": "github",
        "message": "Connection timeout after 5 seconds",
        "code": "API_TIMEOUT",
        "timestamp": "2026-04-09T19:00:00Z",
        "recoveryAction": "Retrying in 60 seconds; using cached
        GitHub PR data"
      }
    ],
    "datasources": {
      "hq": { "status": "success", "lastSync":
        "2026-04-09T19:00:00Z" },
```

```
      "github": { "status": "failed", "lastSync":
        "2026-04-09T18:55:00Z" }
    }
  },
  "duration_ms": 5200,
  "message": "Report generated with partial data (GitHub API
        unavailable)"
}
```

cURL Examples:

```
# Synchronous refresh
curl -X POST \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"fullRefresh": true, "mode": "sync"}' \
  https://myxxit-devopshq.onrender.com/api/monitoring/refresh |
        jq .

# Asynchronous refresh (returns immediately)
curl -X POST \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -d '{"mode": "async"}' \
  https://myxxit-devopshq.onrender.com/api/monitoring/refresh |
        jq .

# With max age check (return cached if <5 min old)
curl -X POST \
  -H "Authorization: Bearer YOUR_API_KEY" \
  -d '{"maxAge": 300}' \
  https://myxxit-devopshq.onrender.com/api/monitoring/refresh |
        jq .
```

---

3. GET /api/monitoring/reports/{id}

Fetch a previously generated report by ID.

Request:

```
GET /api/monitoring/reports/550e8400-e29b-41d4-a716-446655440000
HTTP/1.1
Host: myxxit-devopshq.onrender.com
Authorization: Bearer YOUR_API_KEY
```

Response 200 OK:

```
{
  "status": "success",
  "report": {
    "id": "550e8400-e29b-41d4-a716-446655440000",
    "generatedAt": "2026-04-09T19:00:00Z",
    ...
```

```
  },
  "retrievedAt": "2026-04-09T19:05:30Z",
  "age_seconds": 330
}
```

Response 404 Not Found:

```
{
  "status": "error",
  "code": "REPORT_NOT_FOUND",
  "message": "Report not found (may have expired or been
       deleted)",
  "reportId": "550e8400-e29b-41d4-a716-446655440000"
}
```

cURL Example:

```
curl -H "Authorization: Bearer YOUR_API_KEY" \
  https://myxxit-devopshq.onrender.com/api/monitoring/reports/
       550e8400-e29b-41d4-a716-446655440000 | jq .
```

---

4. GET /api/monitoring/reports

List recent reports with pagination.

Query Parameters:

Parameter	Type	Default	Description
`limit`	number	10	Max reports per page (1-100)
`offset`	number	0	Skip first N reports
`sort`	enum	“generatedAt”	Sort field: “generatedAt” or “createdAt”
`order`	enum	“desc”	“asc” or “desc”
`source`	enum	all	Filter by source: “manual” or “webhook”

Request:

```
GET /api/monitoring/reports?
limit=10&offset=0&sort=generatedAt&order=desc HTTP/1.1
Host: myxxit-devopshq.onrender.com
Authorization: Bearer YOUR_API_KEY
```

Response 200 OK:

```
{
  "status": "success",
```

```
  "reports": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "generatedAt": "2026-04-09T19:00:00Z",
      "refreshSource": "manual",
      "metrics": {
        "totalTasks": 12,
        "blockedCount": 2,
        "riskSummary": { "high": 1, "medium": 2, "low": 9 }
      },
      "createdBy": "selym-api",
      "datasources": {
        "hq": "success",
        "github": "success"
      }
    },
    {
      "id": "550e8400-e29b-41d4-a716-446655440001",
      "generatedAt": "2026-04-09T18:55:00Z",
      ...
    }
  ],
  "pagination": {
    "total": 47,
    "limit": 10,
    "offset": 0,
    "hasMore": true
  }
}
```

cURL Example:

```
curl -H "Authorization: Bearer YOUR_API_KEY" \
  "https://myxxit-devopshq.onrender.com/api/monitoring/reports?
        limit=20&offset=0" | jq .
```

Authentication

Bearer Token Format

All endpoints (except `/health`) require authentication via Bearer token:

```
Authorization: Bearer YOUR_API_KEY
```

Where to get the API key: 1. Set in environment variable `DEVOPS_HQ_API_KEY` (on deployment platform) 2. Use in client applications: `Authorization: Bearer <value>`

Testing Authentication

```
# Valid token
curl -H "Authorization: Bearer valid-api-key" \
  https://myxxit-devopshq.onrender.com/api/monitoring/health
# Response: 200 OK

# Invalid token
curl -H "Authorization: Bearer invalid-key" \
  https://myxxit-devopshq.onrender.com/api/monitoring/refresh
# Response: 401 Unauthorized

# Missing token
curl https://myxxit-devopshq.onrender.com/api/monitoring/refresh
# Response: 401 Unauthorized
```

Rate Limiting

Limits per API Key: - 10 refreshes per minute - 100 report fetches per minute - 1000 requests per hour

Response Headers:

```
X-RateLimit-Limit: 10
X-RateLimit-Remaining: 9
X-RateLimit-Reset: 1712694060
```

When limit exceeded (429 Too Many Requests):

```
{
  "status": "error",
  "code": "RATE_LIMIT_EXCEEDED",
  "message": "Too many requests. Limit: 10 per minute",
  "retryAfter_seconds": 45
}
```

Error Codes Reference

Code	HTTP	Meaning	Recovery
`INVALID_REQUEST`	400	Bad request (invalid parameters)	Check request format
`AUTH_FAILED`	401	Missing or invalid API key	Provide valid Bearer token
`FORBIDDEN`	403		Contact admin

Code	HTTP	Meaning	Recovery
		Valid token but insufficient permissions	
`REPORT_NOT_FOUND`	404	Report ID doesn't exist	Check report ID
`RATE_LIMIT_EXCEEDED`	429	Too many requests	Wait and retry
`INTERNAL_SERVER_ERROR`	500	Server-side error	Retry; contact support
`SERVICE_UNAVAILABLE`	503	Service temporarily down	Retry with exponential backoff
`API_TIMEOUT`	503	Datasource API timed out	Returned partial report
`DATABASE_ERROR`	500	Database connectivity issue	Retry; check service health

Response Structure

All responses follow a consistent structure:

```
{
  "status": "success" | "error" | "degraded",
  "code": "OPERATION_CODE",
  "message": "Human-readable message",
  "data": { ... },
  "errors": [ ... ],
  "timestamp": "ISO8601",
  "requestId": "uuid"
}
```

Fields:

Field	Type	Always Present?	Description
`status`	enum	Yes	Overall result: "success", "error", or "degraded"
`code`	string	No	Machine-readable operation code
`message`	string	Errors only	Human-readable error message

Field	Type	Always Present?	Description
`data`	object	No	Response payload (varies by endpoint)
`errors`	array	No	List of errors if `status` is “error” or “degraded”
`timestamp`	string	Yes	ISO8601 timestamp of response
`requestId`	string	Yes	Unique request ID for debugging

WebHook Integration (Optional Future)

For automated refresh on task/PR updates:

POST /webhooks/hq-task-updated

```
{
  "event": "task.updated",
  "task": {
    "id": "task-001",
    "status": "in-progress",
    "updatedAt": "2026-04-09T19:00:00Z"
  }
}
```

POST /webhooks/github-pr-event

```
{
  "event": "pull_request.opened",
  "action": "opened",
  "number": 42,
  "pullRequest": {
    "title": "feat: add authentication",
    "state": "open"
  }
}
```

Pagination

List endpoints support cursor-based pagination:

```
# First page
curl -H "Authorization: Bearer KEY" \
```

```
  "https://...?limit=10&offset=0"

# Next page (offset += limit)
curl -H "Authorization: Bearer KEY" \
  "https://...?limit=10&offset=10"

# Last page (check hasMore flag)
```

Performance Notes

Operation	Typical Time	Max Time
Full refresh	1-2s	10s (with timeout fallback)
Cached report	<50ms	50ms (in-memory)
Report listing	100-300ms	2s (database query)
Health check	50-200ms	2s (API pings)

Client SDK Usage

JavaScript/Node.js

```
import axios from 'axios';

const client = axios.create({
  baseURL: 'https://myxxit-devopshq.onrender.com',
  headers: {
    Authorization: `Bearer ${process.env.DEVOPS_HQ_API_KEY}`
  }
});

// Refresh
const { data: report } = await client.post('/api/monitoring/
        refresh', {
  fullRefresh: true,
  mode: 'sync'
});
console.log(`Total tasks: ${report.report.metrics.totalTasks}`);

// List reports
const { data: list } = await client.get('/api/monitoring/reports?
        limit=10');
list.reports.forEach(r => console.log(`${r.id}: $
        {r.metrics.blockedCount} blocked`));
```

Python

```python
import requests

API_KEY = os.environ['DEVOPS_HQ_API_KEY']
BASE_URL = 'https://myxxit-devopshq.onrender.com'

headers = {'Authorization': f'Bearer {API_KEY}'}

# Refresh
response = requests.post(
    f'{BASE_URL}/api/monitoring/refresh',
    json={'fullRefresh': True},
    headers=headers,
    timeout=10
)
report = response.json()['report']
print(f"Total tasks: {report['metrics']['totalTasks']}")

# Get specific report
response = requests.get(
    f'{BASE_URL}/api/monitoring/reports/{report_id}',
    headers=headers
)
print(response.json()['report'])
```

cURL

See examples throughout this document.

Testing the API

```bash
# 1. Test health (no auth needed)
curl https://myxxit-devopshq.onrender.com/api/monitoring/health

# 2. Test refresh (requires API key)
API_KEY="your-key-here"
curl -X POST \
  -H "Authorization: Bearer $API_KEY" \
  https://myxxit-devopshq.onrender.com/api/monitoring/refresh |
        jq .

# 3. Check specific report
curl -H "Authorization: Bearer $API_KEY" \
  https://myxxit-devopshq.onrender.com/api/monitoring/reports/
        {REPORT_ID} | jq .

# 4. List recent reports
```

```
curl -H "Authorization: Bearer $API_KEY" \
  "https://myxxit-devopshq.onrender.com/api/monitoring/reports?
        limit=5" | jq .
```

Support & Debugging

Having issues?

1. Check `/api/monitoring/health` first (might be service outage)
2. Verify API key is valid and in `Authorization: Bearer` format
3. Check request body for parameter errors
4. Look at `errors` array in response (may contain datasource failures)
5. Retry with `fullRefresh: true` to bypass cache

Debugging tips:

```
# Verbose output
curl -v -X POST \
  -H "Authorization: Bearer $API_KEY" \
  https://myxxit-devopshq.onrender.com/api/monitoring/refresh |
        jq .

# Check response headers
curl -i https://myxxit-devopshq.onrender.com/api/monitoring/health

# Test with full JSON pretty-printing
curl -s ... | jq '.'
```

API Version: 1.0
Last Updated: 2026-04-09
Status: Specification Complete, Ready for Implementation